TOOLS & SOFTWARE

Git & GitHub

A practical, interview-ready tour of Git and GitHub — from staging and branches to rebasing, PRs, and rescuing yourself when things go wrong.

git · github · version-control · branching · merge-rebase · pull-requests · workflow

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Git Is and Why It Matters
2. Setup and Getting Started
3. The Core Loop: Status, Stage, Commit
4. Branches: Cheap, Fast, Essential
5. Merge vs Rebase
6. Remotes, Push, Pull, Fetch
7. Resolving Merge Conflicts
8. Ignoring Files with .gitignore
9. Tags and Releases
10. GitHub Flow and Pull Requests
11. Undoing Mistakes
12. Worked Mini-Example: A Feature from Scratch to Merged
13. Cheat-Sheet Reference
14. Best Practices
15. Common Pitfalls
16. Interview Questions & Answers
17. Further Learning

What Git Is and Why It Matters

Git is a distributed version control system (VCS): it records snapshots of your project over time, lets many people work in parallel, and makes it safe to experiment because nothing is ever truly lost. GitHub is a hosting platform built on top of Git that adds collaboration — pull requests, reviews, issues, and CI/CD.

Every developer touches Git daily, and interviewers probe it because fluency signals you can collaborate on real codebases without creating chaos. The key mental shift: Git does not store diffs by

default — it stores **snapshots** of your entire tree, with unchanged files stored as pointers to identical content. Each snapshot (a *commit*) is identified by a SHA-1/SHA-256 hash of its content, which is why history is tamper-evident.

> **Distributed**
> Unlike older centralized systems (SVN, CVS), every clone is a *full* repository with complete history. You can commit, branch, diff, and view logs entirely offline; the network is only needed to share.

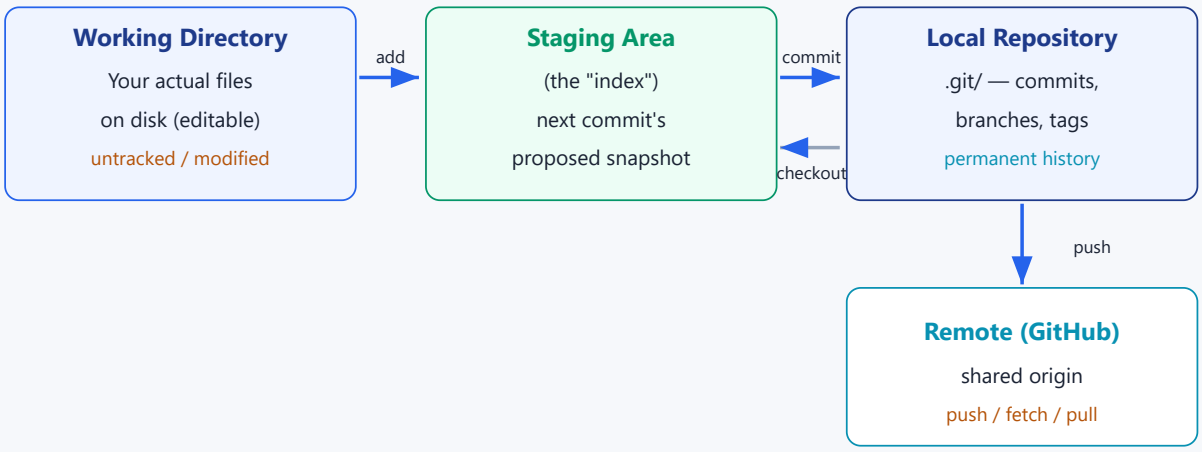


Fig 1. The three local states plus the remote. Files flow right as you add → commit → push, and left as you checkout → pull.

Setup and Getting Started

Install Git, then set your identity — these values are stamped into every commit you author.

BASH

```bash
# Verify / install
git --version

# One-time global identity (used in every commit)
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.com"

# Sensible defaults
git config --global init.defaultBranch main   # name the first branch "main"
git config --global pull.rebase false          # merge on pull (explicit is good)
git config --global core.editor "code --wait"  # use VS Code for messages

# See everything (and where it came from)
git config --list --show-origin
```

Create a repo two ways: start fresh, or copy an existing one.

BASH

```bash
# Start a brand-new repo in the current folder
git init

# Or clone an existing remote (creates a folder, downloads full history)
git clone https://github.com/owner/repo.git
git clone git@github.com:owner/repo.git   # SSH form (needs an SSH key)
```

✦ **SSH keys**

Generate once with `ssh-keygen -t ed25519 -C "ada@example.com"`, add the `.pub` key to GitHub → Settings → SSH keys. SSH avoids typing credentials and is the standard for push access.

The Core Loop: Status, Stage, Commit

The everyday rhythm is a tight loop. `git status` is your compass — run it constantly.

BASH

```
git status                # what's changed, staged, untracked
git add file.py           # stage one file
git add .                 # stage everything in current dir (respects .gitignore)
git add -p                # interactively stage selected hunks (great for clean commi
git commit -m "Add login validation"
git commit -am "Fix typo"  # stage all TRACKED files + commit in one step
git log --oneline --graph --all   # compact, visual history
```

> ⚠ **-am skips new files**
>
> `git commit -am` only stages files Git already tracks. Brand-new (untracked) files still need an explicit `git add` first.

Anatomy of a commit

A commit stores: the snapshot (a tree of file contents), author + committer + timestamps, a message, and the hash(es) of its parent commit(s). Chaining parents forms the history graph.



Fig 2. Each commit points back to its parent. A branch is just a lightweight, movable pointer to one commit.

Branches: Cheap, Fast, Essential

A branch is literally a 41-byte file containing a commit hash. Creating one is instant. Branches let you isolate work — a feature, a bugfix, an experiment — without disturbing `main`.

BASH

```
git branch                      # list local branches (* = current)
git branch feature/login        # create (does not switch)
git switch feature/login        # switch to it (modern)
git switch -c feature/login     # create AND switch
git checkout -b feature/login   # older equivalent of the line above

git switch main                 # go back
git branch -d feature/login     # delete (safe: refuses if unmerged)
git branch -D feature/login     # force delete
git branch -m old new           # rename
```

`HEAD` is the pointer to "where you are" — usually it points at a branch, which points at a commit. Checking out a commit directly puts you in *detached HEAD* (commits you make there belong to no branch and can be lost).

✦ **switch vs checkout**

Modern Git split the overloaded `checkout` into two verbs: `git switch` for branches and `git restore` for files. They're clearer and harder to misuse — prefer them.

Merge vs Rebase

Both integrate changes from one branch into another, but they produce different histories. This is a top interview topic.

MERGE

Creates a **merge commit** with two parents, tying the branches together. History is non-linear but *truthful* — it records exactly what happened. Non-destructive.

BASH

```
git switch main
git merge feature/login
```

REBASE

Replays your branch's commits one-by-one *on top of* the target, creating new commits with new hashes. History is **linear and clean**, but you rewrote it.

BASH

```
git switch feature/login
git rebase main
```

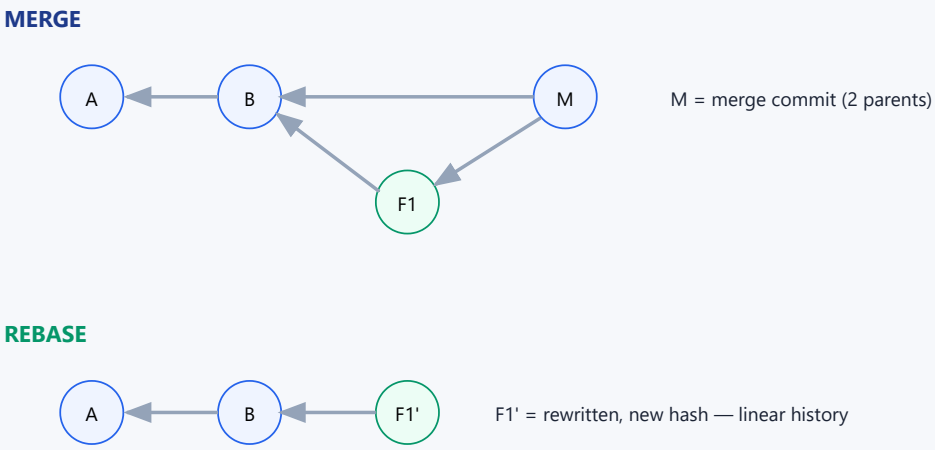


Fig 3. Merge preserves the branch topology with a join commit; rebase rewrites feature commits onto the tip of main for a straight line.

The Golden Rule of Rebase

Never rebase commits that others have already pulled (anything pushed to a shared branch like `main`). Rebasing rewrites hashes; collaborators' histories diverge and merges become a nightmare. Rebase only *local, private, unpushed* work.

Interactive rebase — polish before you share

BASH

```
git rebase -i HEAD~4   # edit the last 4 commits
# In the editor, change the verb before each commit:
#   pick    - keep as-is
#   reword  - change the message
#   squash  - fold into previous commit (keep both messages)
#   fixup   - fold into previous, discard this message
#   drop    - delete the commit
#   edit    - pause to amend content
```

Remotes, Push, Pull, Fetch

A *remote* is a named URL to another copy of the repo. The default is `origin`. Remote-tracking branches like `origin/main` are your local cache of where the remote was at your last fetch.

BASH

```
git remote -v                          # list remotes
git remote add origin git@github.com:owner/repo.git
git push -u origin main                # push + set upstream (first time)
git push                               # thereafter

git fetch origin                       # download new commits, DON'T merge
git pull                               # = fetch + merge (or rebase) into current bran
git pull --rebase                      # fetch then rebase (linear local history)
```



✦ **Fetch, then look**

`git pull` blends two steps and can surprise you. `git fetch` + `git log origin/main` lets you inspect incoming changes before integrating — safer on busy teams.

Resolving Merge Conflicts

A conflict happens when two branches change the same lines (or one edits a file the other deleted). Git pauses and marks the file with conflict markers.

TEXT

```
<<<<<<< HEAD
color = "blue"        # your current branch's version
=======
color = "green"       # the incoming branch's version
>>>>>>> feature/theme
```

Resolve by editing the file to the final desired content, removing all three marker lines, then staging and continuing.

BASH

```
git status                # lists "both modified" files
#  ... edit each conflicted file by hand or with a mergetool ...
git add resolved_file.py  # mark as resolved
git commit                # finish a merge (message pre-filled)
# If you were rebasing instead:
git rebase --continue
git merge --abort         # bail out entirely, restore pre-merge state
git rebase --abort
```

> **Reduce conflicts**
> Pull/rebase from `main` frequently, keep branches short-lived and small, and agree on formatting (a shared formatter kills whitespace conflicts). Enable `git rerere` to auto-reuse recorded conflict resolutions.

Ignoring Files with .gitignore

A `.gitignore` file lists patterns Git should not track — build artifacts, dependencies, secrets, OS cruft. Commit it so the whole team shares the rules.

BASH

```
# .gitignore — one pattern per line
node_modules/       # directory (trailing slash)
*.log               # all .log files
dist/               # build output
.env                # secrets — NEVER commit these
.DS_Store           # macOS junk
!keep.log           # ! negates: force-track this one
/config.local       # leading / anchors to repo root only
```

> **Already-tracked files are not ignored**
> `.gitignore` only affects *untracked* files. If you already committed `.env`, add the pattern **and** run `git rm --cached .env`. If a secret was pushed, rotate it immediately — it lives in history forever until the history is rewritten.

Tags and Releases

Tags mark a specific commit, typically a release version. Use annotated tags (they store author, date, message, and can be signed) for releases.

BASH

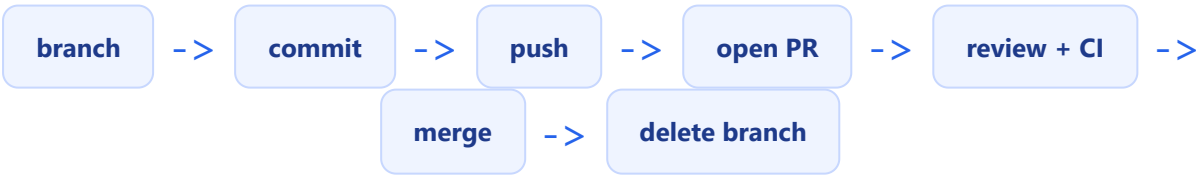
```bash
git tag v1.4.0                      # lightweight (just a pointer)
git tag -a v1.4.0 -m "Release 1.4.0" # annotated (recommended)
git tag                             # list
git show v1.4.0                     # inspect
git push origin v1.4.0              # tags are NOT pushed by default
git push origin --tags              # push all tags
git tag -d v1.4.0                   # delete locally
```

✦ **SemVer**

Follow Semantic Versioning: `MAJOR.MINOR.PATCH` — bump MAJOR for breaking changes, MINOR for backward-compatible features, PATCH for bug fixes. On GitHub, a tag can be promoted to a *Release* with notes and binaries.

GitHub Flow and Pull Requests

GitHub Flow is a simple, widely used branching model: `main` is always deployable; all work happens on short-lived branches merged back via **pull requests (PRs)**.



A PR is a request to merge your branch, with a diff, discussion, line comments, and automated checks (CI/CD). Reviewers approve or request changes; merging is often gated on green checks and approvals.

BASH

```
# Typical PR workflow with the GitHub CLI (gh)
git switch -c feature/search
#  ... work, commit ...
git push -u origin feature/search
gh pr create --title "Add search" --body "Implements fuzzy search"
gh pr status          # see your PRs
gh pr checks          # CI status
gh pr merge --squash --delete-branch
```

Merge strategies on GitHub

Strategy	Result	Use when
Merge commit	Keeps all commits + a merge commit	You want full, truthful history
Squash and merge	Collapses the PR into one commit on main	Feature branches with messy WIP commits (most common)
Rebase and merge	Replays commits, no merge commit	You want linear history and clean per-commit changes

Fork-based contribution (open source)



BASH

```
git clone git@github.com:you/project.git
git remote add upstream git@github.com:original/project.git
git fetch upstream
git switch -c fix/typo upstream/main
#  ... commit, push to YOUR fork, open PR against upstream ...
git push origin fix/typo
```

Undoing Mistakes

Git rarely loses data — knowing the right undo tool turns panic into a two-command fix. The right choice depends on *where* the change lives (working dir, staged, committed, pushed).

Goal	Command	Notes
Discard unstaged file changes	`git restore file`	Destroys uncommitted edits
Unstage (keep edits)	`git restore --staged file`	Reverse of `git add`
Fix the last commit	`git commit --amend`	Rewrites it — only if unpushed
Move branch back, keep changes staged	`git reset --soft HEAD~1`	Undo commit, keep work
Move back, keep changes unstaged	`git reset HEAD~1`	Default (mixed)
Nuke commits AND changes	`git reset --hard HEAD~1`	Destructive — careful
Undo a pushed commit safely	`git revert <sha>`	New commit that inverts it
Recover "lost" commits	`git reflog`	Your safety net

✕ **reset vs revert**

`reset` rewrites history (moves the branch pointer) — safe only on unpushed, private commits. `revert` adds a new commit that undoes a previous one — the correct tool for anything already shared, because it doesn't rewrite history others depend on.



Fig 4. reset moves the branch pointer back (history changes); revert leaves history intact and appends a commit that cancels the target.

Stash — park work temporarily

BASH

```
git stash                 # shelve all tracked changes, clean the working dir
git stash -u              # include untracked files
git stash push -m "wip: half-done nav"   # named stash
git stash list            # stash@{0}, stash@{1}, ...
git stash pop             # re-apply newest and remove it from the stack
git stash apply stash@{1} # re-apply a specific one, keep it in the stack
git stash drop stash@{0}  # delete one
```

The reflog safety net

BASH

```
# Accidentally reset --hard and lost commit C?
git reflog                # shows every HEAD movement with its sha
git reset --hard a3f9c21  # jump back to that recorded state
# reflog entries persist ~90 days by default — almost nothing is truly gone
```

Worked Mini-Example: A Feature from Scratch to Merged

A complete, realistic flow adding a feature, keeping it current, and shipping it via a PR.

BASH

```
# 1. Start from an up-to-date main
git switch main
git pull

# 2. Branch for the feature
git switch -c feature/user-avatars

# 3. Work in small, meaningful commits
git add src/avatar.js
git commit -m "Add Avatar component with fallback initials"
git add src/avatar.css
git commit -m "Style avatar sizes and ring"

# 4. main moved on while you worked — integrate it
git fetch origin
git rebase origin/main        # replay your 2 commits on the latest main
#   (resolve any conflicts, then: git add <file> && git rebase --continue)

# 5. Clean up the history before sharing (optional)
git rebase -i origin/main     # squash the two commits into one tidy commit

# 6. Push and open a PR
git push -u origin feature/user-avatars
gh pr create --fill

# 7. After approval + green CI, squash-merge and clean up
gh pr merge --squash --delete-branch
git switch main && git pull
```

Cheat-Sheet Reference

Task	Command
Clone a repo	`git clone <url>`
Check state	`git status`
Stage all	`git add .`
Commit	`git commit -m "msg"`
New branch + switch	`git switch -c name`
Merge branch into current	`git merge name`
Rebase onto main	`git rebase main`
Push first time	`git push -u origin name`
Update from remote	`git pull --rebase`
See history graph	`git log --oneline --graph --all`
Diff working vs staged	`git diff` / `git diff --staged`
Undo pushed commit	`git revert <sha>`
Recover lost work	`git reflog`
Shelve work	`git stash` / `git stash pop`
Who changed this line?	`git blame file`
Find the bad commit	`git bisect start`

Best Practices

✓ **Commit small and often**

Each commit should be one logical change that builds and passes tests. Small commits are easier to review, revert, and bisect.

✓ **Write good messages**

Imperative subject under ~50 chars ("Add rate limiting"), blank line, then a body explaining *why*. Conventional Commits (`feat:` , `fix:` , `docs:`) enable automated changelogs and versioning.

✦ **Branch per task**

Short-lived branches with descriptive names (`feat/` , `fix/` , `chore/`) keep `main` clean and PRs focused.

✦ **Protect main**

On GitHub, enable branch protection: require PR reviews, passing CI, and up-to-date branches before merge. Never force-push shared branches.

Common Pitfalls

⚠ **Committing secrets**

API keys and `.env` files leak constantly. Add them to `.gitignore` up front, use secret scanning, and rotate anything exposed — rewriting history alone is not enough.

⚠ **Giant commits**

"Fix everything" commits are unreviewable and un-revertable. Stage selectively with `git add -p` .

✕ **force-push on shared branches**

`git push --force` can erase teammates' commits. If you must, use `--force-with-lease` , which refuses if the remote moved since your last fetch.

⚠ **Confusing fetch and pull**

`pull` auto-merges and can create surprise merge commits or conflicts mid-flow. Prefer `fetch` + inspect, or set `pull.rebase true` .

Interview Questions & Answers

What's the difference between `git merge` and `git rebase`?

Merge integrates branches by creating a merge commit with two parents, preserving the true, non-linear history. Rebase replays your commits on top of the target branch, producing new commits with new hashes and a clean linear history. Merge is safe for shared branches; rebase should only be used on local, unpushed commits (the Golden Rule) because it rewrites history.

Explain the three areas: working directory, staging area, repository.

The working directory holds your actual editable files. The staging area (index) holds the proposed contents of your next commit — you choose exactly what goes in with `git add`. The repository (`.git`) stores committed snapshots permanently. Changes flow working → staged → committed.

When would you use `reset` vs `revert`?

Use `reset` to move a branch pointer backward on *local, unpushed* commits (`--soft` keeps staged, `--mixed` keeps unstaged, `--hard` discards). Use `revert` for commits already pushed/shared: it creates a new commit that inverts the target, so no history is rewritten and collaborators are unaffected.

What is `HEAD`, and what is a detached HEAD?

`HEAD` is a pointer to your current location — normally it points to a branch, which points to a commit. A detached HEAD occurs when you check out a commit (or tag) directly; new commits you make belong to no branch and can be garbage-collected unless you create a branch to keep them.

How do you resolve a merge conflict?

Run `git status` to find conflicted files. Open each, locate the `<<<<<<<` / `=======` / `>>>>>>>` markers, edit to the correct final content, and delete the markers. Then `git add` the resolved files and `git commit` (or `git rebase --continue`). Use `--abort` to bail out.

What does `git cherry-pick` do?

It applies the change introduced by a specific commit onto your current branch as a new commit. Useful for backporting a single bugfix to a release branch without merging the whole feature branch: `git cherry-pick <sha>`.

How would you find which commit introduced a bug?

`git bisect`: mark a known-good and known-bad commit, and Git binary-searches the range, checking out midpoints for you to test (`git bisect good` / `bad`). In O(log n) steps it pinpoints the first bad commit. For a single line, `git blame` shows who last changed it.

What's the difference between `git fetch` and `git pull`?

`fetch` downloads new commits and updates remote-tracking branches (e.g. `origin/main`) but does *not* change your working branch. `pull` is `fetch` followed by an automatic `merge` (or `rebase`) into your current branch. Fetch is non-destructive and lets you inspect before integrating.

You committed to the wrong branch. How do you fix it?

If unpushed: note the commit sha, `git switch correct-branch`, `git cherry-pick <sha>`, then return and `git reset --hard HEAD~1` on the wrong branch to remove it. Alternatively `git reset --soft HEAD~1` to uncommit (keeping changes staged), stash, switch, and re-commit.

What is `git stash` and when is it useful?

It shelves your uncommitted changes and reverts the working directory to a clean HEAD, storing them on a stack. Useful when you must switch branches urgently (e.g. a hotfix) but your current work isn't ready to commit. Restore with `git stash pop` (apply + remove) or `apply` (apply + keep).

Why are Git commit hashes content-addressable, and why does that matter?

A commit's hash is derived from its content (tree, parents, author, message). Any change to history produces different hashes downstream, making tampering detectable and guaranteeing integrity. It's also why rebasing/amending creates new commits rather than editing in place.

What is `--force-with-lease` and why prefer it over `--force`?

After a rebase you must force-push to update a remote branch. Plain `--force` overwrites the remote unconditionally and can erase a teammate's commits pushed since you last fetched. `--force-with-lease` only succeeds if the remote is still where you last saw it, acting as a safety check against clobbering others' work.

Further Learning

- Read *Pro Git* (Chacon & Straub) — free online; the authoritative, approachable reference.
- Practice interactively at **Learn Git Branching** (visualizes branches, merge, and rebase).
- Master the **GitHub CLI** (`gh`) and GitHub Actions for CI/CD on your PRs.
- Explore advanced tools: `git bisect`, `git worktree`, `git submodule`, and `git rerere`.
- Adopt **Conventional Commits** + semantic-release for automated versioning and changelogs.
- Learn to read the plumbing: `git cat-file`, `git rev-parse`, and how objects (blob/tree/commit/tag) form the DAG.